

Network Graph

Multiple Inputs

Custom Training

model.**fit**(X, Y)

trainNetwork(X, Y)



Software Engineering & Data Science
SEIS 766 Vision A.I.
Dr. Chih Lai

Outline

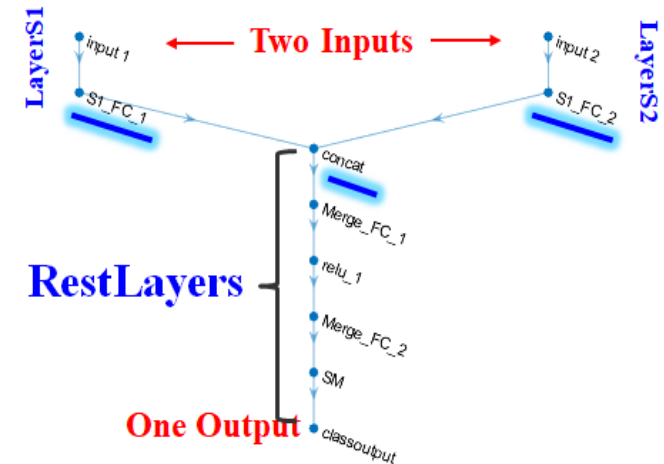
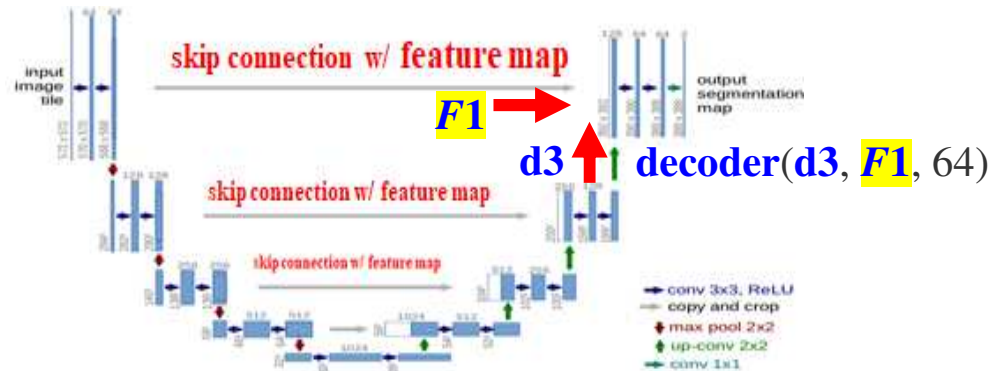
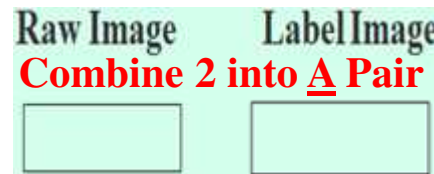
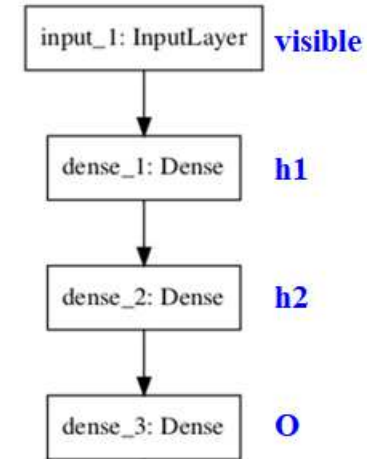
➡ Programming More Advanced / Complex NN Architecture

• Functional API Graph Connect

■ ONE networks with 2 (or more) Inputs

• Functional API

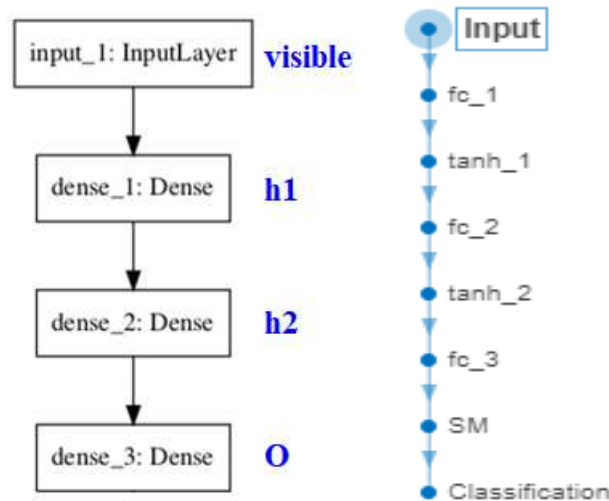
■ Prepare *Data* for ONE Network with 2 (or more) Inputs



■ Build your own *training loop* & loss function... for training **multi-NNs** together

Why “*Functional API*” / “*Graph Connect*”?

■ Ordinary NN



<https://keras.io/getting-started/faq/#how-can-i-obtain-the-output-of-an-intermediate-layer>

```
visible = Input(shape=(13,))
```

```
hidden1 = Dense(9, activation='relu', name='h1')(visible)
```

```
hidden2 = Dense(5, activation='relu', name='h2')(hidden1)
```

```
output = Dense(2, activation='softmax', name='O')(hidden2)
```

```
# creating original model
```

```
model_1 = Model(inputs = visible, outputs = output)
```

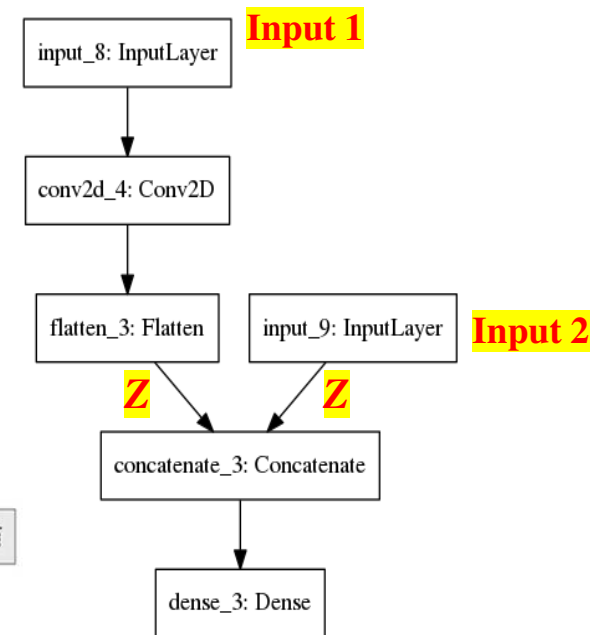
```
model.fit(X, Y, epochs=150, batch_size=15) # training the model here
```

```
classes = model_1.predict(X)
```

■ *Function API* allows more flexibility in building more advanced NN

with 2 inputs into 1 NN OR

even w/ 2 NNs (i.e. GAN) ...later

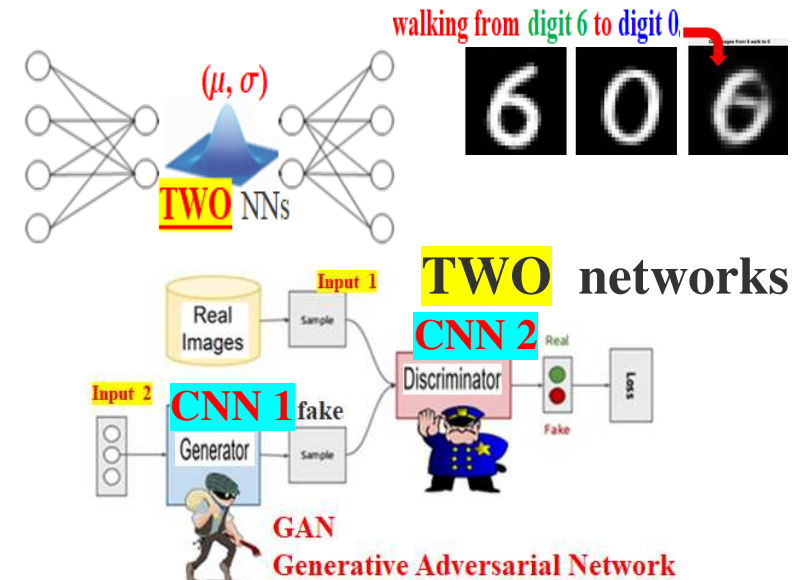
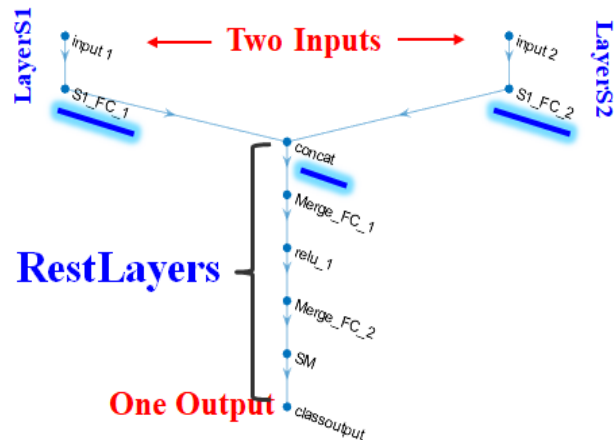


Outline

- Program More Advanced / Complex NN Architecture

- **Functional API** **Graph Connect**

➔ **ONE** networks with **2** (or more) Inputs



- Prepare *Data* for **ONE** Network with **2** (or more) Inputs
- Build your own *training loop* & **loss** function... for training **multi-NNs** together

Keras Multiple Input Sources, Example

```
from keras.layers import Input, Concatenate, Conv2D, Flatten, Dense
from keras.models import Model

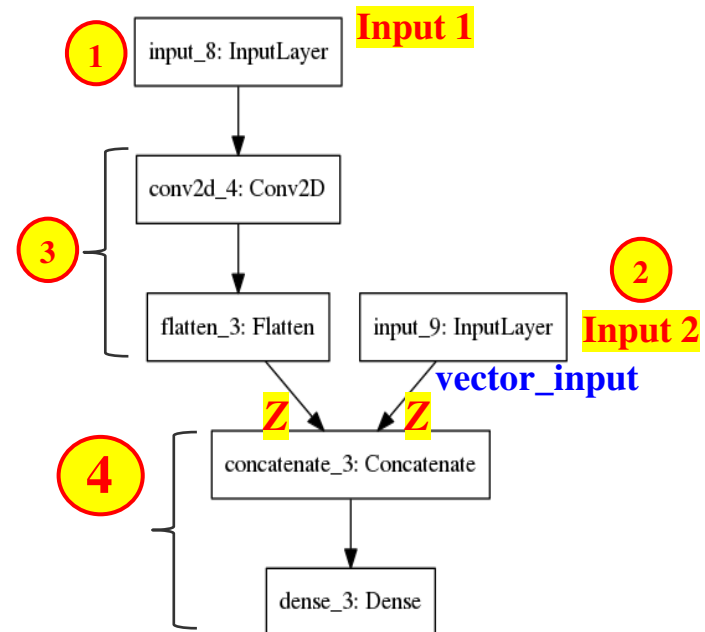
# Define two input layers
image_input = Input((32, 32, 3)) 1
vector_input = Input((6,)) 2

# Convolution + Flatten for the image
conv_layer = Conv2D(32, (3,3))(image_input) 3
flat_layer = Flatten()(conv_layer)

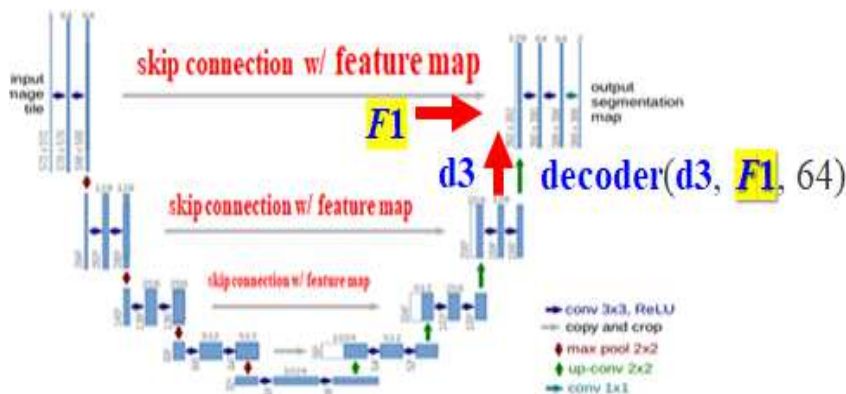
# Concatenate the convolutional features and the vector input
concat_layer = Concatenate()([vector_input, flat_layer]) 4
output = Dense(3)(concat_layer)

# define a model with a list of two inputs
model = Model(inputs=[image_input, vector_input], outputs=output)
```

<https://stackoverflow.com/questions/53516662/two-inputs-to-one-model-in-keras>



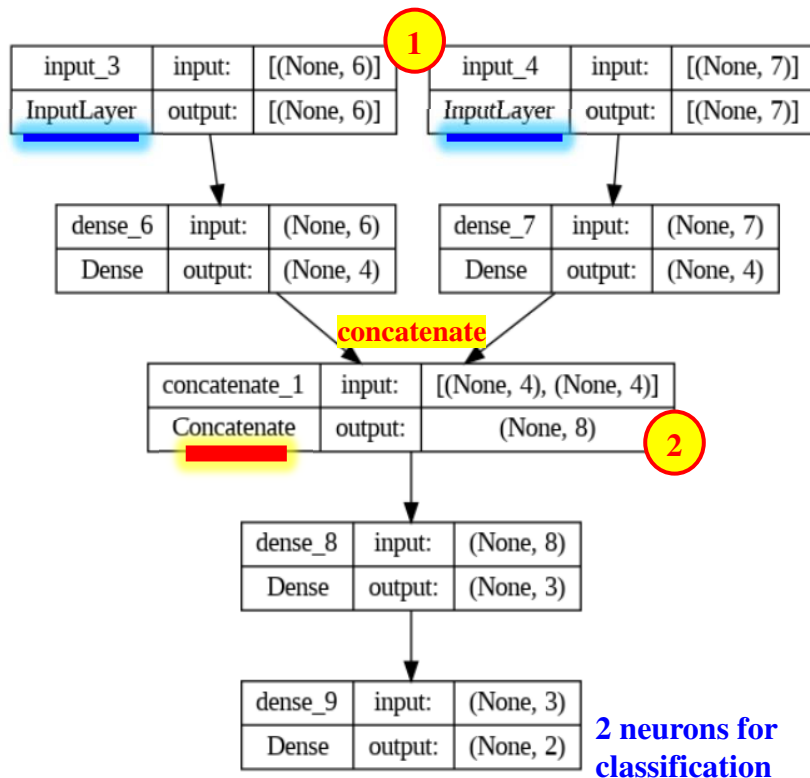
https://keras.io/api/utils/model_plotting_utils/



Layer (type)	Output Shape	Param #	Connected to
input_8 (InputLayer) 1	(None, 32, 32, 3)	0	
conv2d_4 (Conv2D) 3	(None, 30, 30, 32)	896	input_8[0][0]
input_9 (InputLayer) 2	(None, 6)	0	
flatten_3 (Flatten)	(None, 28800)	0	conv2d_4[0][0]
concatenate_3 (Concatenate) 4	(None, 28806)	0	input_9[0][0] flatten_3[0][0]
dense_3 (Dense)	(None, 3)	86421	concatenate_3[0][0]
Total params: 87,317			
Trainable params: 87,317			
Non-trainable params: 0			

Concatenate 2 Inputs in NN Via Functional API

- Homework: cell DNA 13-D data breaks into 3 sets: {X1 6-D, X2 7-D, and Y}
- How do I know all layers are correctly connected / concatenated? `tf.keras.utils.plot_model`
- <https://pyimagesearch.com/2019/02/04/keras-multiple-inputs-and-mixed-data/>



VAI_Slide_Own_TrainLoop_Loss\
VAI_Slide_2_Inputs.ipynb

```
from keras.models import Model
from keras.layers import Input, Dense, concatenate
```

1. Define two input layers

```
input1 = Input(shape=(input1_shape,))
```

```
input2 = Input(shape=(input2_shape,))
```

few more dense layers on each side

2. Concatenate the inputs

```
merged_inputs = concatenate([input1, input2], axis=-1)
```

Additional layers after concatenation

```
dense1 = Dense(64, activation='relu')(merged_inputs)
```

```
dense2 = Dense(32, activation='relu')(dense1)
```

```
output = Dense(output_shape, activation='softmax')(dense2)
```

Create the model

```
model = Model(inputs = [input1, input2], outputs = output)
```

Compile the model

```
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

Print the model summary

```
model.summary()
```

Outline

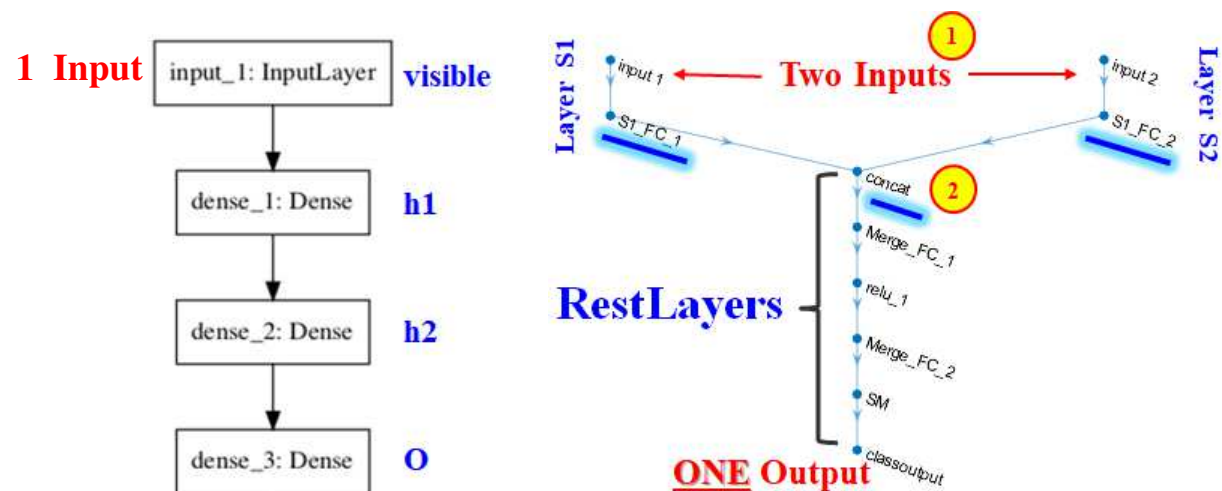
- Program More Advanced / Complex NN Architecture

- **Functional API** **Graph Connect**

- ONE networks with **2** (or more) Inputs

- Functional API Graph Connect

- **Prepare data** for ONE Network with **2** (or more) Inputs



- Build your own *training loop* & loss function... for training multi-NNs together

Prepare 2 Streams of Data X & Y ... and Consistently Split 0.8 / 0.2

- Homework: cell DNA **13-D** data breaks into **3** sets: {**X1 6-D**, **X2 7-D**, and **Y**}

```
data1 = pd.read_csv('data1.csv') # also z-score
data2 = pd.read_csv('data2.csv') # also z-score
In_Y = pd.read_csv('targets.csv')
YCat = to_categorical(In_Y, 2)
model.fit([data1, data2], YCat) # NO split
```

here we still use **“fit()”** in Keras

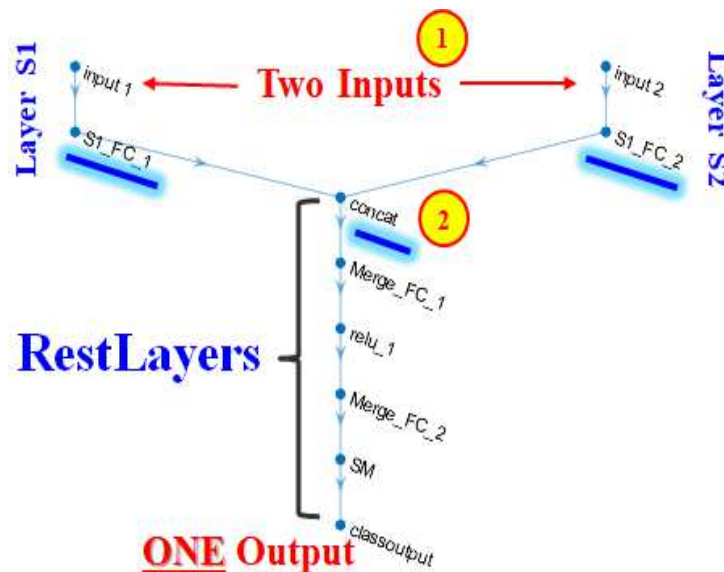
→ Make sure split two inputs consistently

split TOGETHER

```
train1, test1, train2, test2, train_targets, test_targets =
    train_test_split(data1, data2, YCat, test_size=0.2)
```

STILL use Keras fit() function

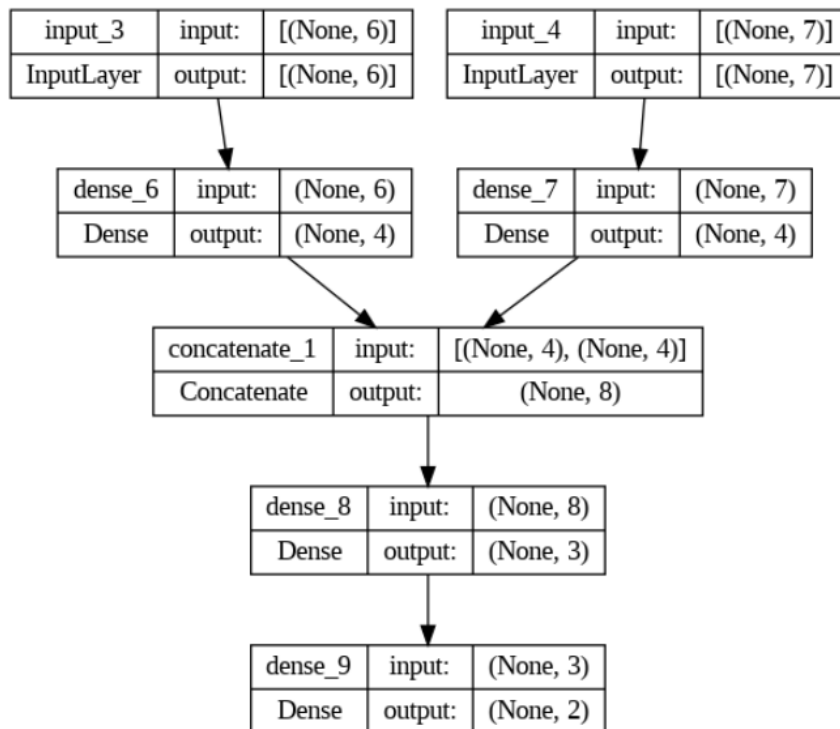
```
model.fit([train1, train2], train_targets)
```



```
data1 = readmatrix('data1.csv'); % also z-score
data2 = readmatrix('data2.csv'); % also z-score
In_Y = readmatrix('targets.csv');
Y = categorical(In_Y);
% may split first
% [TrainIdx, TestIdxs] = splitlabels(dsAll, [0.8 0.2],
inputs = {data1, data2}; % Two input sources
targets = Y;
[net, trainInfo] = trainnet(inputs, targets, lgraph, "BEC", options);
```

Concatenate 2 Inputs, Homework NN Architecture

- **Homework:** cell DNA **13-D** data breaks into **3** sets: {**X1 6-D**, **X2 7-D**, and **Y**}
 1. Train your NN using “**fit()**” or “**trainNetwork()**” and show confusion matrix generated from this training
 2. Train NN using **your own** “**training loop**” and **your own** “**binary cross entropy**” loss function. Show the confusion matrix from this training. You are ****NOT**** allowed to train your NN using the “**fit()**” or “**trainNetwork()**” function.



```
data1 = pd.read_csv('data1.csv') # also z-score
data2 = pd.read_csv('data2.csv') # also z-score
In_Y = pd.read_csv('targets.csv')
YCat = to_categorical(In_Y, 2)

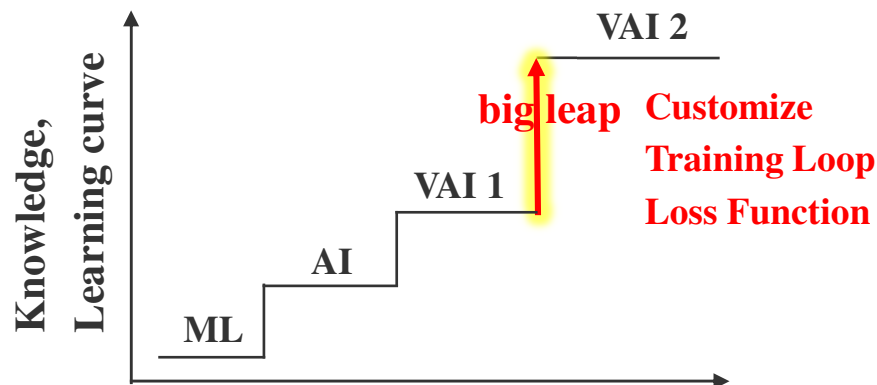
# split TOGETHER
train1, test1, train2, test2, train_targets, test_targets =
    train_test_split(data1, data2, YCat, test_size=0.2)

model.compile(optimizer="adam", loss=my_loss)
# STILL use Keras fit() function
model.fit([train1, train2], train_targets)
```

Outline

- Program More Advanced / Complex NN Architecture
 - **Functional API** **Graph Connect**
- **ONE** networks with **2** (or more) Inputs
 - Functional API Graph Connect
- Prepare *Data* for **ONE** Network with **2** (or more) Inputs

➔ Build your own training loop & loss function... for training **multi-NNs** together

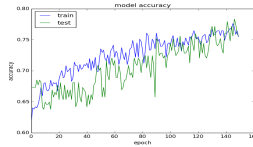


model.**fit**(X, Y)
trainNetwork(X, Y)

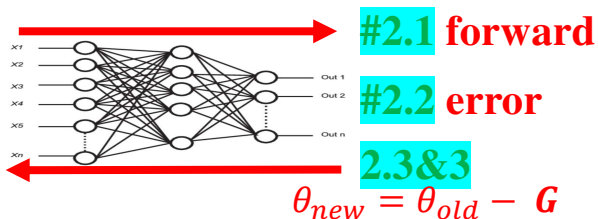


Pseudo Code for Customize **Training Loop** w/ Custom **Loss Function**

- Create **batch** queue
- Create plot monitor (if needed)



- **Loop** epochs yourself **# 0.**
 - **Shuffle** batch queue
 - **Get** next batch **# 1.**
 - Update learning rate (if needed)
 - Compute **loss** **# 2.**
 - **Forward** to get batch predictions **# 2.1**
 - Compute error based on **your need** **# 2.2**
 - **Backward** update θ s in **net** **# 2.3** **# 3.**
 - Update training history plot



Create batch queue **mbq**

Loop epochs yourself **# 0.**

shuffle(mbq)

X = **next**(mbq) **# 1.** get next batch

lossVal, gradient = **Loss**(net, X, Y,) **# 2.1** **2.2** **2.3**

update θ s in **net** **# 3.**

End

function **Loss** **# 2.**

Prediction = **forward**(net, X, Y,) **# 2.1**

lossVal = **MSE/CrossEntropy/...**(Prediction, Y) **# 2.2**

gradient of **lossVal** against **net. θ s** **# 2.3**



model.fit(X, Y_train, nb_epoch=50, batch_size=4)

[net, trainInfo] = **trainNetwork**(ZX, Y, lgraph, options);

Implementation

VAI_Slide_Own_TrainLoop_XEntropy.ipynb

- **GradientTape** watches & records operations in forward pass and loss computation, so TF can compute gradients during backprop in reverse order of those operations (chain rule)

Epoch Loop

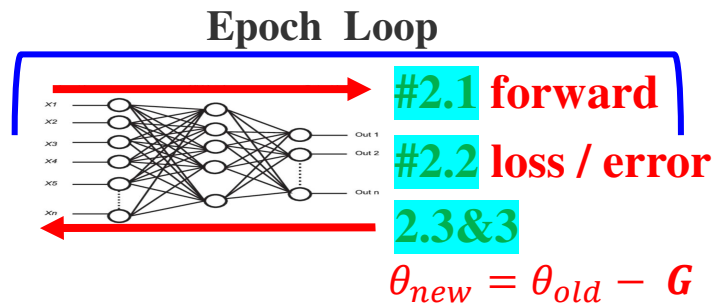
```

with tf.GradientTape() as tape:
    y_pred = your_model(x, training=True) #2.1 Forward pass: model predicts 1 BAT
    loss = your_loss_fn(y_true, y_pred) #2.2 Compute error or loss

    G = tape.gradient(loss, your_model.trainable_variables) #2.3
    optimizer.apply_gradients(zip(G, your_model.trainable_variables)) #3
    
```

■ Loop epochs yourself # 0.

- Shuffle batch queue
- Get next batch # 1.
- Update learning rate (if needed)
- Compute **loss** # 2.
 - Forward to get batch predictions # 2.1
 - Compute error based on **your need** # 2.2
 - Backward update θ s in net # 2.3 # 3.



pytorch

```

y_pred = model(x) #2.1
loss = loss_fn(y_pred, y_true) #2.2
optimizer.zero_grad()
loss.backward() #2.3
optimizer.step() #3
    
```

$$J(w_{ij}) = (a_k(\sum_k(w_{jk}, a_j(\sum_j(w_{ij}))))))$$

$$G = \frac{\partial J}{\partial w_{ij}} = \frac{\partial J}{\partial a_k} \cdot \frac{\partial a_k}{\partial \sum_k} \cdot \frac{\partial \sum_k}{\partial a_j} \cdot \frac{\partial a_j}{\partial \sum_j} \cdot \frac{\partial \sum_j}{\partial w_{ij}}$$

Keras, Customize Training Loop w/ Custom Loss Function

BAT_size = 128 MaxEpochs = 10

create batches

train_dataset = tf.data.Dataset.from_tensor_slices(x_train, y).shuffle(buffer_size=50).batch(BAT_size)

model = keras.Model(In_Layer, Out_Layer)

for epoch in range(MaxEpochs): # 0.

 epoch_loss = 0.0 BAT_count = 0

 # Iterate all batches in dataset, bat_step = 0, 1, 2...

 for bat_step, (xBat, yBat) in enumerate(train_dataset): # 1.

 # train 1 batch

 loss_value = train_BAT(model, optimizer, YourLoss_fn, xBat, yBat) # 2.

 # Accumulate and print progress

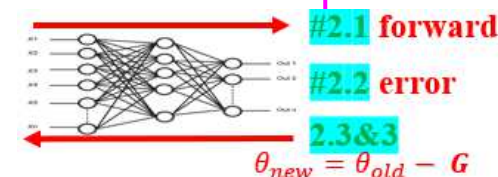
 epoch_loss += loss_value BAT_count += 1

 # Average loss for the epoch & print

 epoch_loss /= batch_count

VAI_Slide_Own_TrainLoop_XEntropy.ipynb

1. Turn **x_train, y** to TF Dataset (GPU ready)
2. **Shuffle** every **buffer_size** records
3. Pack shuffled records into **batches**



def train_BAT(model, optimizer, YourLoss_fn, xBat, yBat):

 # Record steps & orders in compute (x, yhat, loss) for gradient

 with tf.GradientTape() as tape:

 # Forward pass: model predicts on one batch

 yhat = model(x_batch, training=True) # 2.1

 # Compute loss, you need check BAT $\neq$ NONE

 loss_value = YourLoss_fn(y_batch, yhat) # 2.2 θ_s

 # Compute gradient of loss w.r.t θ_s , $G = \partial(\text{loss}) / \partial(\theta_s)$

 Gs = tape.gradient(loss_value, model.trainable_weights) # 2.3

 # Use optimizer to update θ_s , $\theta_{new} = \theta_{old} - G$

 optimizer.apply_gradients(zip(Gs, model.trainable_weights)) # 3.

return loss_value

■ Loop epochs yourself # 0.

● Shuffle batch queue

● Get next batch # 1.

● Update learning rate (if needed)

● Compute loss # 2.

 • Forward to get batch predictions # 2.1

 • Compute error based on your need # 2.2

 • Backward update θ_s in net # 2.3 # 3.

Keras, Customize Training Loop w/ Custom C.E. Loss

Customize Loop

VAI_Slide_Own_TrainLoop_XEntropy.ipynb

```
# https://keras.io/api/optimizers/
# Instantiate an optimizer....
# ***** BUT, NO handle of batch size
optimizer = tf.keras.optimizers.SGD(learning_rate)

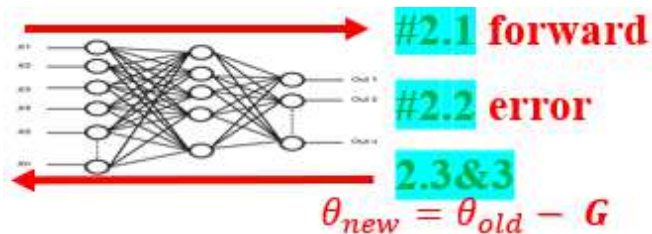
for epoch in range(1000):
    with tf.GradientTape() as tape:
        # 2.1 Forward pass to get current predict
        y_pred = model(X_train)
        # 2.2 call loss func to compute loss
        loss = your_categorical_crossentropy(y_train_one_hot, y_pred)

        # 2.3 Get gradients of loss w.r.t the NN weights  $\theta$ s "dlgradient(loss, net)"
        gradients = tape.gradient(loss, model.trainable_variables) #  $\theta$ s

        # 3. Update weights  $\theta$ s of the NN model "sgdmupdate(net, gradients)"
        optimizer.apply_gradients(zip(gradients, model.trainable_variables))

print(f'Epoch {epoch}/{epochs}, Loss: {loss.numpy()}')
```

```
# Custom cross-entropy loss function for 2-class
def your_categorical_crossentropy(y_true, y_pred):
    epsilon = 1e-15 # Small constant to avoid log(0)
    y_pred = tf.clip_by_value(y_pred, epsilon, 1 - epsilon)
    return -tf.reduce_mean(y_true * tf.math.log(y_pred))
```

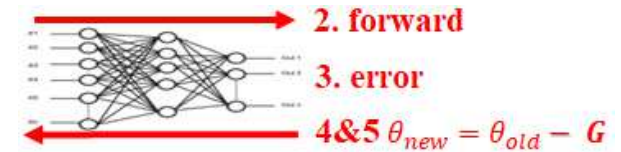


- Loop epochs yourself # 0.
 - Shuffle batch queue
 - Get next batch # 1.
 - Update learning rate (if needed)
 - Compute loss # 2.
 - Forward to get batch predictions # 2.1
 - Compute error based on your need # 2.2
 - Backward update θ s in net # 2.3 # 3.

Appendix-1

Customized Training in **Matlab**, Compare to **Keras**

Customized Training



VAI_Slide_Own_TrainLoop_XEntropy.m

```
while epoch < numEpochs && ~monitor.Stop % epoch loops
    epoch = epoch + 1;

    % Shuffle data.
    shuffle(mbq);

    % Loop over mini-batches.
    while hasdata(mbq) && ~monitor.Stop
        iteration = iteration + 1;
        % Read mini-batch of data.
        [X, T] = next(mbq); % #1.

        % Evaluate model gradients, state, and
        % your loss using dlfeval to update NN, #2. 3. 4.
        [loss, gradients, state] = dlfeval(@yourLoss, net, X, T);
        net.State = state;

        % adjust learning rate for decay
        learnRate = initialLearnRate/(1 + decay*iteration);

        % Update NN \theta_s using the SGDM optimizer, #5.
        [net, velocity] = sgdmupdate(net, gradients, velocity,
                                     learnRate, momentum);
    end
end
```

Loop epochs yourself #0.

- Shuffle batch queue
- Get next batch #1.
- Update learning rate (if needed)
- Compute loss #2.
 - Forward to get batch predictions #2.
 - Compute error based on your need #3.
 - Backward update θ s in net #4. #5.

```
# create batches
train_dataset = tf.data.Dataset.from_tensor_slices(x_train).shuffle(buffer_size=50).batch(BAT_size)

model = keras.Model(In_Layer, Out_Layer)

for epoch in range(MaxEpochs):
    epoch_loss = 0.0    BAT_count = 0

    # Iterate all batches in dataset, bat_step = 0, 1, 2...
    for bat_step, (xBat, yBat) in enumerate(dataset):
        # train 1 batch
        loss_val = train_BAT(model, optimizer, YourLoss_fn, xBat, yBat)

        # Accumulate and print progress
        epoch_loss += loss_val    BAT_count += 1

    # Average loss for the epoch & print
    epoch_loss /= batch_count
```

```
def train_BAT(model, optimizer, YourLoss_fn, xBat, yBat):
    # Record steps & orders in compute (x, yhat, loss) for gradient
    with tf.GradientTape() as tape:
        # Forward pass: model predicts on one batch
        yhat = model(x_batch, training=True)

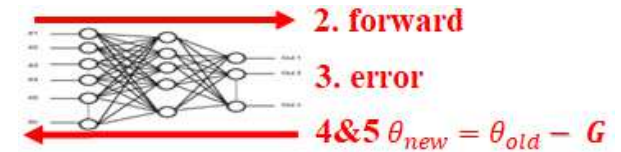
        # Compute loss, you need check BAT \neq NONE
        loss_value = YourLoss_fn(y_batch, yhat)

        # Compute gradient of loss w.r.t \theta_s, G = \partial(loss) / \partial(\theta_s)
        Gs = tape.gradient(loss_value, model.trainable_weights)

        # Use optimizer to update \theta_s, \theta_{new} = \theta_{old} - G
        optimizer.apply_gradients(zip(Gs, model.trainable_weights))

    return loss_value
```

Customize **Loss Function**, Matlab



VAI_Slide_Own_TrainLoop_XEntropy.m

% Loop over epochs.

```
while epoch < numEpochs && ~monitor.Stop
```

```
    epoch = epoch + 1;
```

% Shuffle data.

```
    shuffle(mbq);
```

% Loop over mini-batches.

```
    while hasdata(mbq) && ~monitor.Stop
```

```
        iteration = iteration + 1;
```

% Read mini-batch of data.

```
        [X, T] = next(mbq); % # 1.
```

% Evaluate model gradients, state, and loss using dlfeval and the % your modelLoss func to update network state, # 2. 3. 4.

```
        [loss, gradients, state] = dlfeval(@modelLoss, net, X, T, classes);  
        net.State = state;
```

% adjust learning rate for decay

```
        learnRate = initialLearnRate/(1 + decay*iteration);
```

% Update NN θ s using the SGDM optimizer, # 5.

```
        [net, velocity] = sgdmupdate(net, gradients, velocity, learnRate,  
                                     momentum);
```

% Update the training progress monitor

```
    end  
end
```

• Evaluate loss

- NN forward to get prediction $\hat{y}$ # 2.
- compute loss values, i.e. $Y - \hat{y}$ # 3.
- compute gradient # 4.
- Update θ s # 5.

```
%% T = Target, from (0, 1) becomes (1, 2) automatically  
function [loss, gradients, state] = modelLoss(net, X, T, classes)
```

% Forward data through network.

```
[Yh, state] = forward(net, X); # 2
```

```
% one-hot for the following XEntropy computing  
tt = onehotencode(T, 1, 'ClassNames', 1:classes);
```

% Calculate cross-entropy loss.

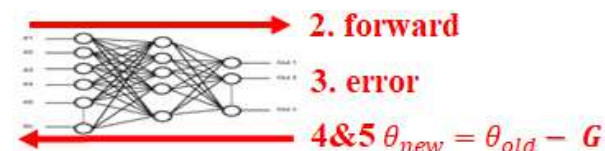
```
loss = crossentropy(Yh, tt); # 3
```

% Calculate gradients of loss w.r.t. parameters θ s

```
gradients = dlgradient(loss, net.Learnables); # 4  
return
```

Customize **Loss Function**, Matlab

VAI_Slide_Own_TrainLoop_XEntropy.m



% Loop over epochs.

while epoch < numEpochs && ~monitor.Stop

epoch = epoch + 1;

% Shuffle data.

shuffle(mbq);

% Loop over mini-batches.

while hasdata(mbq) && ~monitor.Stop

iteration = iteration + 1;

% Read mini-batch of data.

[X, T] = next(mbq); **% # 1.**

**% Evaluate model gradients, state, and loss using dlfeval and the
% your modelLoss func to update network state, # 2. 3. 4.**

[loss, gradients, state] = **dlfeval**(@**modelLoss**, net, X, T, classes);
net.State = state;

% adjust learning rate for decay

learnRate = initialLearnRate/(1 + decay*iteration);

% Update NN θ s using the SGDM optimizer, # 5.

[net, velocity] = **sgdmupdate**(net, gradients, velocity, learnRate,
momentum);

% Update the training progress monitor

end

end

• **Evaluate loss**

- NN **forward** to get prediction $\hat{y}$ **# 2.**
- **compute loss** values, i.e. $Y - \hat{y}$ **# 3.**
- compute gradient **# 4.**

• **Update θ s # 5.**

%% T = Target, from (0, 1) becomes (1, 2) automatically
function [loss, gradients, state] = **modelLoss**(net, X, T, classes)

% Forward data through network.

[Yh, state] = **forward**(net, X); **# 2**

% one-hot for the following XEntropy computing

tt = **onehotencode**(T, 1, 'ClassNames', 1:classes);

% Calculate cross-entropy loss.

loss = **crossentropy**(Yh, tt); **# 3**

% Calculate gradients of loss w.r.t. parameters θ s

gradients = **dlgradient**(loss, net.Learnables); **# 4**

return

$$y \log \frac{1}{1 + e^{-\theta^T x}} + (1 - y) \log \left(1 - \frac{1}{1 + e^{-\theta^T x}}\right)$$

**Binary
Cross
Entropy**

Yh = max(min(**Yh**, 1 - epsilon), epsilon);

loss = -mean(**T** .* log(**Yh**) + (1 - **T**) .* log(1 - **Yh**));